

Section 91 — Primality And Integer Factorization

Beyond the last sieve cell

This lesson derives A–C. Problem D is editorial-only.

A. Deterministic Miller–Rabin for 64 bits

For odd n , write $n-1 = d * 2^s$ with odd d . If n is prime, Fermat plus repeated squaring forces a^d either to be 1, or to reach -1 before reaching 1. A base that violates this is a compositeness witness.

One base only gives a probabilistic statement. For every unsigned 64-bit input, the fixed bases

```
2, 325, 9375, 28178, 450775, 9780504, 1795265022
```

are a proven deterministic witness set. This is a bounded-domain theorem, not a pattern that may be extended to arbitrary integers.

In C++, multiply residues with unsigned 128-bit arithmetic before reducing. Python integers already expand. Trial-divide a few tiny primes first: it removes edge cases and accelerates common composites.

B. Pollard–rho factor discovery

Iterate $x \leftarrow x*x+c \bmod n$. Modulo an unknown prime factor, the sequence eventually cycles. Floyd’s two-speed pointers make $\gcd(|x-y|, n)$ reveal a nontrivial shared factor when the two positions first coincide modulo that factor.

The attempt can return n ; restart with new c and seed. A discovered divisor need not be prime. Recursively split both pieces until Miller–Rabin certifies every leaf, then sort. The invariant is multiplicative: the product of values on the work stack and accepted leaves is always the original number.

C. Multiplicative functions after factorization

From $n = \text{product } p^e$, Euler’s function is obtained once per distinct prime:

```
phi(n) = n * product (p-1)/p.
```

Perform $\text{answer} = \text{answer}/p*(p-1)$ in that order, so the intermediate division is exact. This separates two concerns cleanly: randomized factor discovery and deterministic evaluation from the certified prime multiset.

How to find it

When a bounded sieve is impossible, ask whether the task needs all primes up to a limit or only the factors of a few individual integers. The second shape suggests primality certification plus recursive splitting. Always state the integer domain before selecting witness bases or a multiplication method.

Reading the Miller–Rabin chain

The values

$$a^d, a^{2d}, a^{4d}, \dots, a^{(2^s d)} = a^{(n-1)}$$

are successive squares. For prime n , if this chain finishes at 1, the first occurrence of 1 is either the initial value or is preceded by -1 : a field has only the two square roots 1 and -1 of 1. Therefore a nontrivial square root of 1, or a chain that never reaches the required pattern, certifies compositeness. Passing all selected bases is not itself a factorization.

Pollard–rho failure is an expected event

The gcd can be 1 for many iterations, a proper divisor, or n . The last case means the two walks became indistinguishable modulo all factors at once; it does not mean n is prime. Restart with a different polynomial or seed. Once a proper divisor d appears, recursively process both d and n/d because either may still be composite.

Use absolute difference before gcd and a multiplication routine safe for the full input width. Handle $n=1$, even inputs, prime powers, semiprimes with close factors, and repeated factors. For reproducible debugging, allow the random seed to be fixed; randomness affects discovery time, not the required answer.

Independent validation

On small integers, compare primality with trial division and factorization with an SPF table. For large outputs, verify every reported leaf with Miller–Rabin and multiply all leaves back using a wide type. Then compare multiplicative statistics with direct gcd counting only on small n ; this separates bugs in factor discovery from bugs in the prime-power formula.

Exercises

A–C develop the primality/factorization pipeline. D transfers the factor multiset to two new multiplicative statistics.